Progreso del curso · 32 capítulos, 9 partes

Parte 0
Introducción

1 **2**

Parte I
Fundamentos estructurales y de datos

3 4 5◇ 6 7 8 9

Parte II
Estimación flexible y primer despliegue

10 11 12 13 14 15

Parte III
Diseños de identificación con ML

16◇ 17◇ 18◇

Parte IV
Publicación y operación en producción

19 20

Parte V
Heterogeneidad, políticas y targeting

21 22

Parte VI
Explicación, equidad y decisión

23 24 25 26

Parte VII
Métodos avanzados

27◇ 28◇

Parte VIII
Estructura, monitoreo y frontera

29 30 31 32

previos · actual · próximos · ◇ profundización electiva (no bloquea el camino ⋆)

2

CAPÍTULO 2

★ Camino crítico

Anatomía de un sistema de ML en producción

Parte 0 · Sesión S2

Plantilla B (ingeniería y producción)

IA Causal en Producción

Curso ECO-490 / ECON-540 · Edición 2026

Roadmap del capítulo

Objetivos de aprendizaje

1. Enumerar los componentes de un sistema de ML más allá del algoritmo (Huyen, 2022).
2. Explicar por qué reentrenar es más delicado en un sistema causal que en uno predictivo.
3. Dejar el esqueleto de repositorio del curso funcionando localmente (Lab 0).

⋆ Antes de clase

Lectura: Cap. 2 — Anatomía del sistema (@cap-anatomia-sistema)
Lab: L02
Entrega/hito: Lab 0

Agenda

01 · El problema de diseño: modelo entrenado ≠ sistema en producción

02 · Model-centric vs. system-centric; proceso lineal vs. iterativo

03 · Decisión: los 4 requisitos DMLS en versión causal

04 · Esqueleto de repositorio del curso

05 · Qué cambia cuando el «modelo» estima un efecto

El problema de diseño

Decisión	Tratar el modelo como un componente de un sistema, no como el sistema entero.
Alternativas	Enfoque centrado en el modelo (model-centric) vs. enfoque centrado en el sistema (system-centric, DMLS).
Trade-off clave	Velocidad de iteración académica vs. confiabilidad operativa.
Stack	Python, Git, Docker, cuenta en Hugging Face Hub (Lab 0).
Riesgo principal	Confundir "el modelo entrena bien" con "el sistema está listo para producción".

Opciones y trade-offs

Model-centric

Mejorar especificación/algoritmo con datos y entorno fijos. Rápido de iterar; ignora todo lo anterior y posterior al modelo entrenado.

System-centric (DMLS)

Repartir el foco entre datos, modelo e infraestructura. Requiere invertir en formato, feature stores, logging, tests — el único enfoque que sobrevive datos y usuarios reales que cambian con el tiempo.

Proceso lineal vs. iterativo

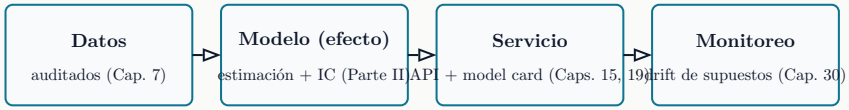
Waterfall asume que el mundo no cambia tras el despliegue; DMLS asume que sí y construye monitoreo para detectarlo.

Decisión adoptada

Enfoque centrado en el sistema + proceso iterativo DMLS, con 4 requisitos reinterpretados en versión causal: confiabilidad del efecto, escalabilidad del pipeline, mantenibilidad de la identificación, adaptabilidad ante drift de supuestos.

Un sistema causal necesita, además de lo que exige DMLS a cualquier sistema, que el efecto siga siendo válido bajo los supuestos declarados — no solo que el servicio esté "arriba".

Sistema causal: proceso iterativo, no lineal



el monitoreo retroalimenta a los datos: el proceso es iterativo (DMLS), no un pipeline de una sola pasada

El esqueleto de repositorio del curso

```
proyecto-causal/
├── data/                # datos crudos y procesados (no versionados en git salvo muestras pequeñas)
├── notebooks/
│   ├── serie_a_caso_estudio/     # A1–A4: identificación y estimación
│   └── serie_b_produccion/       # B1–B4: despliegue en Hugging Face
├── src/
│   ├── identification/   # DAGs, diagnósticos de no confusión y soporte común
│   ├── estimation/       # DML, meta-learners, bosques causales
│   └── service/          # función de inferencia, app FastAPI
├── tests/                # pruebas de humo y de sanidad estadística
├── .github/workflows/    # CI/CD (se detalla en el Capítulo 20)
├── Dockerfile
└── model_card.md         # plantilla del Apéndice G
```

💡 Lab 0: dejar este esqueleto funcionando localmente, con Docker verificado (docker run hello-world).

Recordatorio del caso

Un sistema predictivo puede mostrar una predicción puntual sin más. Un sistema causal responsable debe comunicar el intervalo de confianza y, cuando sea posible, la población para la cual el efecto fue estimado.

Resultado numérico

Tres cambios: (1) el artefacto servido incluye los supuestos de identificación, no solo los coeficientes; (2) la interfaz comunica algo más que un número; (3) reentrenar puede cambiar silenciosamente la población representada.

Lectura de política · Ninguno de los tres tiene análogo obligatorio en un sistema puramente predictivo — son específicos de que el objetivo sea un efecto causal.

Nivel PCH	**Estimando / métrica principal**
Transversal (infraestructura para operar cualquier nivel de la PCH)	Los 4 requisitos DMLS-causal: confiabilidad / escalabilidad / mantenibilidad / adaptabilidad

Pregunta activa (1 min)

Si el estimador es impecable pero nadie puede auditar sus supuestos en producción, ¿está realmente listo?

Escalera PCH: L1 (asociación) $\subseteq$ L2 (intervención) $\subseteq$ L3 (contrafactual) — cada nivel exige supuestos estrictamente más fuertes que el anterior. Si solo predijéramos, ¿a quién trataríamos? Con el efecto, ¿cambia la decisión?

Señales de alerta y checklist operativo

- ☐ El notebook de estimación no tiene ninguna versión de datos ni código asociados.
- ☐ La función de features en entrenamiento y en producción no son literalmente la misma (riesgo de training-serving skew).
- ☐ No existe ningún documento que declare los supuestos de identificación asumidos.
- ☐ No hay mecanismo, ni manual, para detectar si las covariables de producción se parecen a las de entrenamiento.

Si dos o más señales están presentes, el sistema no está listo según los criterios de este curso.

Punto de quiebre · Aplicado

★ De los cuatro requisitos (confiabilidad, escalabilidad, mantenibilidad, adaptabilidad), ¿cuál es más fácil de descuidar en un proyecto real? Justifique.

⏱ 2-4 min · Votación mano alzada o Mentimeter

Discusión y ejercicio

Pregunta de discusión

¿Por qué "reentrenar con datos nuevos" es una decisión más delicada en un sistema causal que en uno puramente predictivo?

Ejercicio (5-10 min)

Complete el Lab 0: esqueleto de repositorio, Git inicializado, cuenta en Hugging Face Hub, Docker instalado y verificado. Entregue una captura de "docker run hello-world" corriendo exitosamente.

Cierre del capítulo

Resumen

- Un sistema de ML tiene componentes más allá del algoritmo: interfaz, datos, backend, infraestructura de despliegue/monitoreo (Huyen, 2022).
- Un sistema causal añade: supuestos de identificación viajando con el artefacto, comunicación de incertidumbre, y cautela al reentrenar.
- El esqueleto de repositorio del curso se usa sin cambios en las Series A y B.

Referencias clave

(1) Huyen (2022), **Designing Machine Learning Systems**, Caps. 1-2

(2) Sculley et al. (2015), «Hidden Technical Debt in Machine Learning Systems»

(3) Breck et al. (2017), «The ML Test Score»

Próximo episodio · Cap. 3 — SCM y Jerarquía Causal de Pearl

Gancho: Con el sistema listo, falta el lenguaje formal para razonar sobre el efecto que ese sistema va a operar.